Test case generation and prioritization: a process-mining approach

Andrea Janes
Free University of Bozen-Bolzano, Piazza Domenicani 3, Bolzano, Italy 39100
Email: ajanes@unibz.it

***Abstract*—Test cases are an essential tool in software quality assurance: they ensure that code behaves as specified in the requirement. However, writing test cases does not have only benefits, it comes with a cost: the programmer has to formulate the test cases and maintain them when the tested source code changes. Particularly for start-ups or small enterprises such costs become prohibitive, which often prefer to invest their time into the development of new functionalities instead of testing. This paper explores the use of process-mining as an approach to create a model of how users interact with a system to a) generate test cases and b) prioritize them. Using process-mining, it is possible to mine from the user behaviour which parts of the system are the most used, in which order they are executed, generate test cases repeating user input, and prioritizing test cases.**

I. Introduction

The term "mining" refers to the process of extracting precious materials, like gold, from a mine, i.e., an excavation or any place that contains such precious materials. Analogously, term "process mining" refers to tools and techniques that, starting from process traces (logs that document events that happened because a process was executed) are able to mine, i.e., extract processes and to support their improvement [1].

The actual mining step in "process mining" constructs a process model that is able to generate the observed data. The mined process can then be used to compare it with an ideal, manually created process model to understand the differences between "what should happen" and "what is really happening"; this activity is called "conformance checking". Based on the mined process model, it is also possible to study the process to identify problems, e.g., bottlenecks, unexpected delays, an unexpected distribution of work, etc.; this activity is called "process improvement".

One possible application of process mining is to log how users interact with software and to mine process models from the collected log files. In a preliminary study, we investigated this possibility [2] and used process mining to mine a model that describes how users navigate through the user interface (UI). To generate the log files, we instrumented an industrial application so that we logged every interaction with the UI, e.g., a button click, editing a text box, etc.

An example of a process map is depicted in Fig. 1. This model depicts typical paths a user traverses when using the software. For example, he or she begins and most often (in the collected data 281 times) starts (opens) a window called "OrderAdministration", which most of the times (232) is immediately closed clicking on "Cancel". (This is because the window "OrderAdministration" contains the shipment date, which the user just wants to consult and then closes the window.)

Using the mined process model, we were able to determine how many and which steps a user needs on average to use a functionality of the application. Based on the performed steps we were able to identify steps that may be merged because the users execute them very often, for example, if a user frequently opens a window to push a button, it would be possible to move that button to the parent window and reduce the amount of work needed to use the software. The decision to restructure UI elements to reduce the effort when working with software depends on more aspects than just the number of clicks to reach a feature, nevertheless, the provided data can be used by developers to support their decisions. Moreover, using the mined process model, it is possible to understand which features of the software are used very rarely and which frequently; a prerequisite is to map software features to the UI components they interact with.

Altogether, process mining resulted to be a viable way to obtain a picture of how the user is using the software. In the next two sections, we explore how to use process mining for test case prioritization and test case generation.

II. Test case prioritization

When executing all test cases takes too long (Extreme Programming recommends 10 minutes as a maximum, otherwise programmers tend to execute them less often [3]), test case prioritization becomes necessary, i.e., to select a reduced number of test cases, which will be executed regularly and which are the most likely to find critical defects in the software. Test cases can be prioritized according to various criteria, we propose to use the value of the feature they test.

According to [4], the perceived value is closely related to usage and the consequences of usage. In the case of software, this would mean that parts of the software that are used more often are indicators of value. The opposite does not hold: something that is rarely used can be also of high value, but intensive usage indicates how immediate the consequences would be of having that feature not working. Often, when a feature is important but used rarely (e.g., the closing of the fiscal year in an accounting software), there is more time to resolve the problem. Test case prioritization based on value should therefore take into account how often a feature is needed, which can be mined based on user behaviour: UI

978-1-5090-6676-6/17 $31.00 © 2017 IEEE
DOI 10.1109/ICSTW.2017.11

IEEE computer society

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 02:01:30 UTC from IEEE Xplore. Restrictions apply.

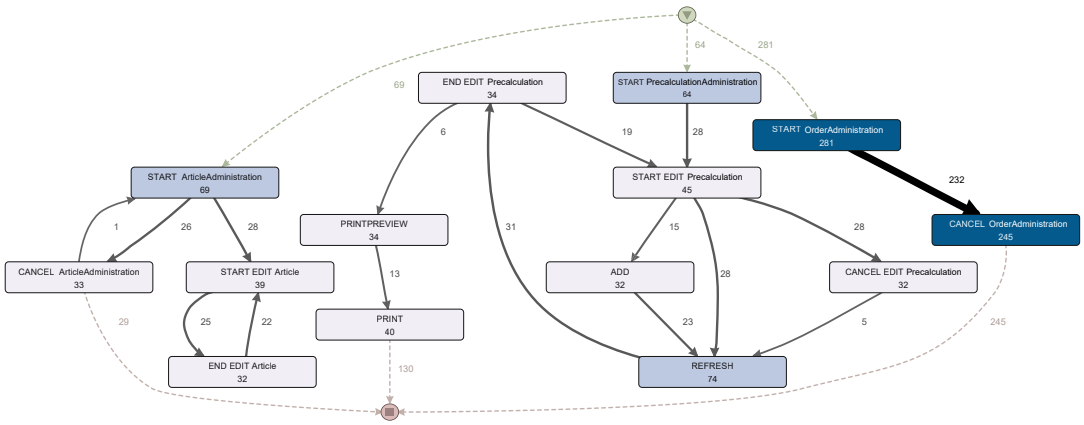

Fig. 1. Process map, showing 5% of the activities and 0% of the paths (adapted from [2]).

elements invoke code, the priority of test cases should depend on the usage intensity of the invoked code they cover.

A manual possibility is to use process mining to accomplish test case prioritization is to manually link test cases and UI elements to features, to mine frequent usage sequences through process mining, and then to prioritize those test cases that test the features identified by process mining. An automated possibility is to calculate the coverage of each test case and to compare it with the coverage of executing an UI action.

III. Test case generation

One approach to generate test cases based on a mined process model is to use the identified user workflows, i.e., typical ways how the user interacts with software, which actions he or she performs, in which sequence actions are performed, etc. to guide the generation of unit test cases. This requires that the collected user interactions are collected at a granularity that allows the re-execution of every user action. Such test cases represent system test cases linked to UI features of the software under test. Technologies that allow the execution of such test cases are e.g., image-recognition-based GUI testing frameworks.

Another approach is to use the mined process model to guide random test generation: random test case generators generate test cases, which call methods of objects with randomly created parameters. The generation process incrementally builds more and longer test sequences by randomly selecting methods and reusing previously generated method sequences (that return objects) as input until a time limit is reached [5].

Random testing uses random input values to evaluate if the tested software is able to perform without unexpected reactions. This approach faces a strong limitation: "random testing may suffer from low code coverage and limited defect detection ability when applied to real-world applications. It is considered unlikely that random approaches are able to exercise all 'deeper' features of a reasonably-sized program by mere chance. These limitations are due to the adoption of a black-box approach without using application-/implementation-specific knowledge [5]."

Generating test cases based on process mining has the potential to overcome these limitations and to radically change the testing landscape: instead of writing test cases, we let users tell us what the most typical test cases should be. Moreover, using the information about how users used previous versions of a software, we want to study how we can use the known input values of the past to generate random tests that obtain a higher code coverage (and therefore a higher defect detection ability) than a purely random based approach.

IV. Conclusions

In addition to the potential applications of process mining, we need to consider the threat to validity of a representational bias, i.e., that we obtain a process model that differs from the real one because of the chosen representation. For example, certain process discovery algorithms cannot represent concurrency, silent actions (in our case activities that occur without that a user uses the UI), or duplicate actions [6]. Process mining provides various opportunities for test case prioritization and generation, but the generated process models need to be validated to minimize representational bias.

References

[1] W. van der Aalst, *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.

[2] S. Astromskis, A. Janes, and M. Mairegger, "A process mining approach to measure how users interact with software: An industrial case study," in *Proceedings of the 2015 International Conference on Software and System Process*. New York, NY, USA: ACM, 2015.

[3] K. Beck and C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd ed. Addison-Wesley, 2004.

[4] R. B. Woodruff, "Customer value: The next source for competitive advantage," *Journal of the Academy of Marketing Science*, vol. 25, no. 2, 1997.

[5] L. Ma, C. V. Artho, C. Zhang, H. Sato, J. Gmeiner, and R. Ramler, "Guiding random test generation with program analysis," in *Software Engineering 2016, Fachtagung des GI-Fachbereichs Softwaretechnik, 23.-26. Februar 2016, Wien, Österreich*, J. Knoop and U. Zdun, Eds., vol. 252. GI, 2016.

[6] W. M. P. van der Aalst, "Process mining: discovering and improving Spaghetti and Lasagna processes," in *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*. IEEE, April 2011.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 02:01:30 UTC from IEEE Xplore. Restrictions apply.